

2. What **write()** Does and Where Bytes Travel

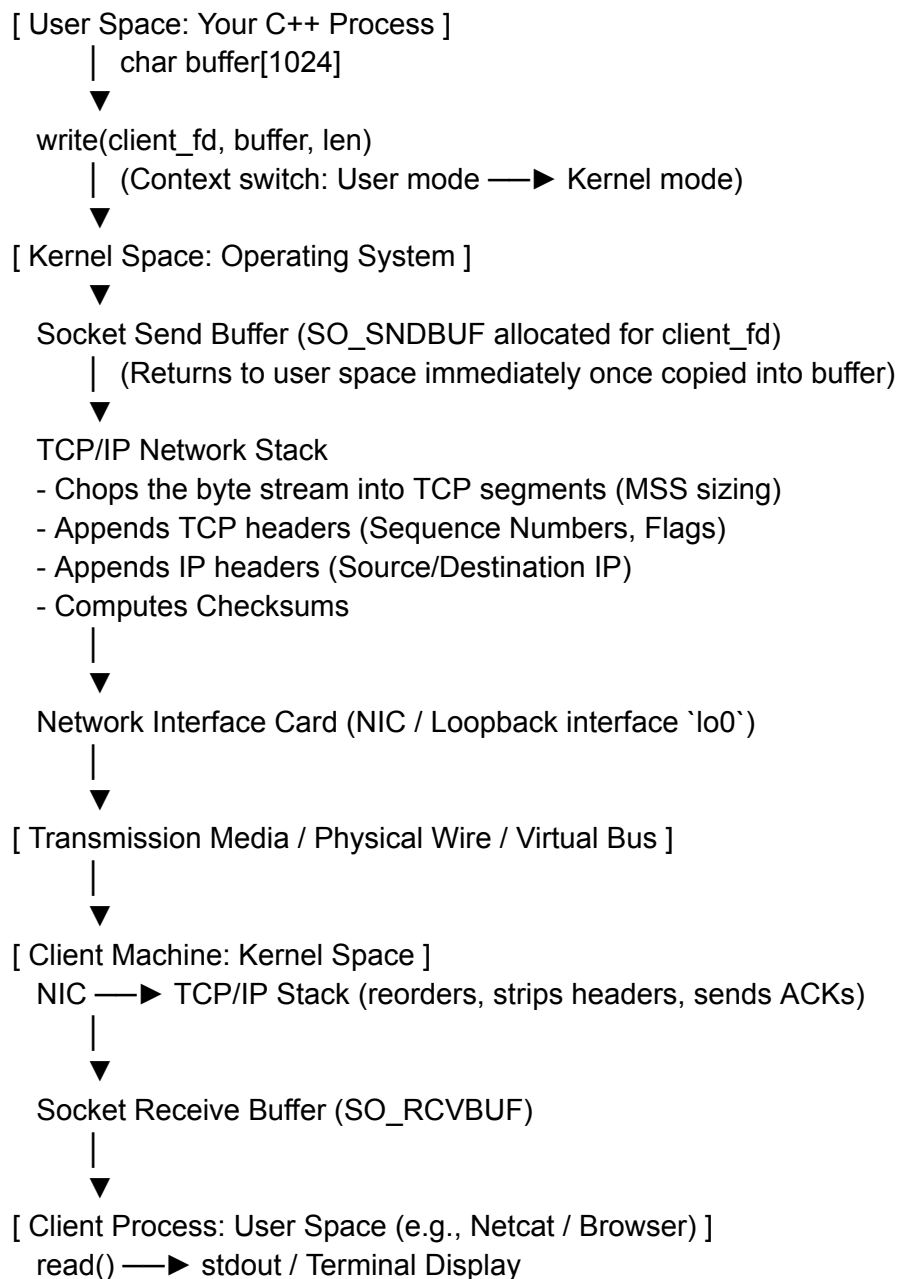
In POSIX socket programming:

C++

```
ssize_t bytes_written = write(client_fd, buffer, bytes_to_write);
```

Path of the Data

write() does **not** write directly to the physical network cable, Wi-Fi interface, or the client's screen. It performs a **memory-to-memory copy** between user space and kernel space:



Core Characteristics of **write()**

1. **Asynchronous with Respect to Network Delivery:** `write()` returns success as soon as the data is copied into the local kernel's `SO_SNDBUF`. It does **not** wait for the destination machine to acknowledge (`ACK`) receipt.
2. **Partial Writes:** If the kernel's send buffer has only 300 bytes of free space and you attempt to write 1024 bytes, the kernel copies 300 bytes and returns `300`. It is the application's responsibility to send the remaining 724 bytes.

3. Byte Stream (TCP) vs. Packet Stream (UDP)

The Underlying Reality: Packet Switching vs. Stream Abstraction

- **Network Layer (IP):** The physical internet is entirely **packet-switched**. Routers handle individual, variable-sized, discrete chunks of data called packets (datagrams).
- **Transport Layer (TCP):** TCP hides the underlying packet-switching mechanism and presents an illusion of a continuous, reliable, bidirectional **byte stream** to the application layer.

Comparison Table

Feature	Byte Stream (TCP / SOCK_STREAM)	Packet Stream (UDP / SOCK_DGRAM)
Mental Model	Continuous garden hose / conveyor belt of bytes	Sealed individual envelopes / discrete boxes
Message Boundaries	None. The OS does not track where one <code>write()</code> ends and another begins.	Preserved. One <code>sendto()</code> corresponds strictly to one <code>recvfrom()</code> .
Delivery Guarantee	In-order, reliable, retransmits lost packets, removes duplicates.	Unreliable, best-effort; packets can arrive out-of-order, duplicated, or not at all.
Flow & Congestion Control	Built-in window sizing, slow start, congestion avoidance.	None. Transmits as fast as the application feeds it.

Application's Role	Must implement message framing (e.g., delimiter bytes or length prefixes).	Reads discrete packets directly; framing is handled by datagram boundaries.
---------------------------	---	---

The "Garden Hose" Behavior (Packet Coalescing & Fragmentation)

If a sender calls:

C++

```
write(fd, "HELLO", 5);
write(fd, "WORLD", 5);
```

- **In a Packet Stream (UDP):**
 - `read()` #1 returns "HELLO" (5 bytes).
 - `read()` #2 returns "WORLD" (5 bytes).
- **In a Byte Stream (TCP):**

The receiver's kernel fills its buffer based on network arrival times, MTU sizes, and sliding windows. The receiver calling `read()` may experience:

 - **Coalescing (One Read):** "HELLOWORLD" (10 bytes).
 - **Fragmentation (Split Reads):** "HEL" (3 bytes), then "LOWORLD" (7 bytes).
 - **Byte-by-Byte:** Ten separate 1-byte reads.

Key Rule for Application Protocols:

Because TCP does not preserve message boundaries, protocols must define their own **framing**:

- **Delimiters:** e.g., HTTP uses `\r\n\r\n` to signify the end of headers.
- **Explicit Length Fields:** e.g., HTTP uses `Content-Length: <N>` to specify the exact payload size.
- **Fixed-Width Framing:** Common in binary protocols where each message or header has a fixed byte size.

4. Handling Partial Writes in POSIX C++

Because a single `write()` may send fewer bytes than requested due to full kernel buffers or signal interruptions, network code should loop until all bytes are sent.

Robust `send_all` Implementation

C++

```
#include <unistd.h>
#include <cstdint>
#include <cerrno>
```

```
/**
```

```
 * Sends all bytes across multiple write() calls if necessary.
```

```
 *
```

```

* @param socket_fd The file descriptor of the connected socket.
* @param data Pointer to the buffer containing data to send.
* @param total_bytes Total number of bytes to transfer.
* @return true on complete delivery, false on disconnection or fatal error.
*/
bool send_all(int socket_fd, const char* data, size_t total_bytes) {
    size_t bytes_sent = 0;

    while (bytes_sent < total_bytes) {
        // Offset the pointer by the number of bytes already written
        ssize_t result = write(socket_fd,
                               data + bytes_sent,
                               total_bytes - bytes_sent);

        if (result < 0) {
            // Interrupted by an OS signal before any bytes were sent; retry
            if (errno == EINTR) {
                continue;
            }
            // EPIPE (Broken pipe: peer closed connection) or other unrecoverable error
            return false;
        }

        if (result == 0) {
            // Peer closed the connection
            return false;
        }

        bytes_sent += static_cast<size_t>(result);
    }

    return true;
}

```

Integration in the Server Loop

```

C++
ssize_t bytes_read = read(client_fd, buffer, sizeof(buffer));
if (bytes_read <= 0) {
    // 0 = EOF (client disconnected cleanly), < 0 = read error
    break;
}

// Ensure the full message is transmitted back
if (!send_all(client_fd, buffer, static_cast<size_t>(bytes_read))) {
    std::cerr << "Failed to send full buffer to client.\n";
    break;
}

```

